

## DOCUMENTACIÓN USO DE IAG EN LAB 01

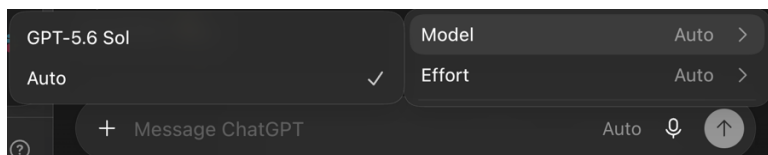
Nicole Eljach

202614655

8 Agosto 2026

Por alguna razón no me estaba funcionando el programa al correr la consola cuando escogía las opciones 1 y 3, después de revisar repetidamente no logré encontrar el error o como solucionarlo. Le pedía a la IA que me ayudara a solucionar el problema. Esta misma me corrigió el tema de poner en minúscula lo de “Book\_tags”, cosa que hice en el momento sin tener presente la actividad del debugger (donde se suponía que nos diéramos cuenta de esto), sin embargo, al momento de llegar al punto del debugger, intencionalmente los cambié nuevamente a “Book\_tags” para poder darme cuenta de cómo funcionaba el proceso.

### Modelo utilizado: ChatGPT (Auto)



### Prompt utilizado:

Podrías analizar mi el error que sale en consola de mi código, que es lo que lo esta causando y como arreglarlo? Primero, aquí está el código de la lógica:

```
import os
```

```
# Importar el modulo de la estructura de datos set
```

```
from DataStructures import set as set
```

```
# Directorio de datos de los archivos
```

```
data_dir = os.path.dirname(os.path.realpath('**file**')) + '/Data/'
```

```
"""
```

La lógica se encarga de procesar los datos y realizar las operaciones sobre ellos. En este caso, se encarga de cargar los datos y realizar las consultas sobre ellos.

```
"""
```

```
def new_logic():
```

```
    """
```

Inicializa los datos y crea un catalogo vacio.

Para esta actividad, un catalogo es un diccionario donde se guardan las estructuras de datos que representan los libros, los autores, los generos y la asociación entre libros y generos.

Se crea un catalogo con las siguientes atributos:

- books: Set para guardar los libros.
- tags: Set para guardar los generos.
- book\_tags: Set para guardar la asociación entre libros y generos.

```
:return: Catalogo inicializado
```

```
:rtype: dict
```

```
"""
```

```
# Creación del catalogo vacio
```

```
catalog = {  
    "books": None,  
    "tags": None,  
    "book_tags": None,  
}
```

```
# Inicialización de las estructuras de datos
```

```
catalog["books"] = set.new_set()  
catalog["tags"] = set.new_set()  
catalog["book_tags"] = set.new_set()
```

```
return catalog
```

```
# Funciones para la carga de datos
```

```
def load_books(catalog, filename):
```

```
    """
```

```
    Función que carga los libros en el catalogo.
```

Por cada libro se toman sus autores y por cada uno de ellos, se crea en la lista de autores, a dicho autor y una

referencia al libro que se esta procesando.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:param filename: Nombre del archivo csv con los libros

:type filename: str

:returns: Tamaño del conjunto de libros

:rtype: int

"""

books = catalog["books"]

booksfile = os.path.join(data\_dir, filename)

catalog["books"] = set.load\_set(books, booksfile)

if empty\_books(catalog):

return 0

else:

return book\_size(catalog)

def load\_tags(catalog, filename):

"""

Carga todos los tags del archivo y los agrega a la lista de tags

:param catalog: Catalogo de la aplicación

:type catalog: dict

:param filename: Nombre del archivo csv con los tags

:type filename: str

:returns: Tamaño del conjunto de tags

:rtype: int

"""

tags = catalog["tags"]

tagsfile = os.path.join(data\_dir, filename)

catalog["tags"] = set.load\_set(tags, tagsfile)

if set.is\_empty(tags):

return 0

else:

return set.size(tags)

def load\_books\_tags(catalog, filename):

"""

Carga los tags de los libros del archivo y los agrega a la lista de tags. Siga el mismo procedimiento que en la carga de libros.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:param filename: Nombre del archivo csv con los tags de los libros

:type filename: str

:returns: Tamaño del conjunto de tags de los libros

:rtype: int

"""

```
# TODO: Mods Lab 1, integrar vista y logica

# Implemente una función que cargue los tags de los libros en el catalogo.

# La función debe recibir el catalogo y el nombre del archivo csv con los tags de los libros.

# La función debe cargar los tags de los libros del archivo y los agregar al conjunto book_tags del catalogo.

# La función debe retornar el tamaño del conjunto de tags de los libros.
```

```
book_tags = catalog.get("Book_tags")

book_tags_file = os.path.join(data_dir, filename)

catalog["Books_tags"] = set.load_set(book_tags, book_tags_file)

if book_tags is None:

    return None

else:

    return set.size(catalog.get("Book_tags"))
```

```
# Funciones de consulta
```

```
def book_size(catalog):

    """

    Obtiene el tamaño del conjunto de libros.

    :param catalog: Catalogo de la aplicación

    :type catalog: dict
```

:returns: Tamaño del conjunto de libros

:rtype: int

"""

books\_size = catalog["book"]

return set.size(catalog["books"])

def tag\_size(catalog):

"""

Obtiene el tamaño del conjunto de tags.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:returns: Tamaño del conjunto de tags

:rtype: int

"""

return set.size(catalog["tags"])

def book\_tag\_size(catalog):

"""

Obtiene el tamaño del conjunto de tags de los libros.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:returns: Tamaño del conjunto de tags de los libros

:rtype: int

"""

return set.size(catalog["book\_tags"])

def empty\_books(catalog):

"""

Verifica si el conjunto de libros esta vacio.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:returns: True si el conjunto de libros esta vacio, False de lo contrario

:rtype: bool

"""

books = catalog["books"]

return set.is\_empty(books)

def empty\_tags(catalog):

"""

Verifica si el conjunto de tags esta vacio.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:returns: True si el conjunto de tags esta vacio, False de lo contrario

:rtype: bool

"""

tags = catalog["tags"]

return set.is\_empty(tags)

def empty\_book\_tags(catalog):

"""

Verifica si el conjunto de tags de los libros esta vacio.

:param catalog: Catalogo de la aplicación

:type catalog: dict

:returns: True si el conjunto de tags de los libros esta vacio, False de lo contrario

:rtype: bool

"""

book\_tags = catalog["book\_tags"]

return set.is\_empty(book\_tags)

aqui el de el view: import sys

import App.logic as logic

"""

La vista se encarga de la interacción con el usuario.

Presenta el menu de opciones y por cada seleccion se hace la solicitud a la lógica para ejecutar la operación solicitada

```
"""
```

```
# Carpeta de datos
```

```
dataFolder = "GoodReads"
```

```
def new_logic():
```

```
    """
```

```
    Función que crea una nueva instancia de la lógica
```

```
    :return: Nueva instancia de la lógica
```

```
    :rtype: logic
```

```
    """
```

```
    app = logic.new_logic()
```

```
    return app
```

```
def print_menu():
```

```
    """
```

```
    Imprime el menú de opciones en consola para el usuario
```

```
    """
```

```
    print("Opciones:")
```

```
    print("1- Cargar Libros")
```

```
print("2- Cargar Tags")  
  
# TODO: Mods Lab 1, agregar la opcion 3.  
  
# Agregue la opción 3 para cargar los tags de los libros.  
  
# Puede guiarse de las opciones 1 y 2.  
  
print("3- Cargar Tags de libros")  
  
print("0- Salir")
```

```
def load_books(app):  
    """  
  
    Función que carga los libros en la aplicación.  
  
    Carga los libros desde el archivo books-small.csv y los almacena en la aplicación  
  
    :param app: Aplicación de la lógica  
    :type app: logic  
    """  
  
    books = logic.load_books(app,  
                              "GoodReads/books-small.csv")  
  
    return books
```

```
def load_tags(app):  
    """  
  
    Función que carga los tags en la aplicación.  
  
    Carga los tags desde el archivo tags.csv y los almacena en la aplicación
```

:param app: Aplicación de la lógica

:type app: logic

"""

```
tags = logic.load_tags(app,  
                        "GoodReads/tags.csv")
```

```
return tags
```

def load\_books\_tags(app):

"""

Función que carga los tags de los libros en la aplicación.

Carga los tags de los libros desde el archivo book\_tags-small.csv y los almacena en la aplicación

:param app: Aplicación de la lógica

:type app: logic

"""

```
booksTags = logic.load_books_tags(app,  
                                   "GoodReads/book_tags-small.csv")
```

```
return booksTags
```

# Se crea la controlador asociado a la vista

```
app = new_logic()
```

# main del ejercicio

```
def main():
```

```
    """
```

```
    Función principal del programa.
```

```
    Presenta el menu de opciones y ejecuta las operaciones solicitadas por el usuario.
```

```
    """
```

```
    working = True
```

```
    # ciclo del menu
```

```
    while working:
```

```
        print_menu()
```

```
        inputs = input("Seleccione una opción para continuar\n")
```

```
        if int(inputs[0]) == 1:
```

```
            print("Cargando información de libros....")
```

```
            books = load_books(app)
```

```
            print("Total de libros cargados: " + str(books) + "\n")
```

```
        elif int(inputs[0]) == 2:
```

```
            print("Cargando información de tags....")
```

```
            tags = load_tags(app)
```

```
            print("Total de tags cargados: " + str(tags) + "\n")
```

```
    # TODO: Mods Lab 1, agregar la funcion opt 3 -> load_book_tags().
```

```
    # Agregue la opción 3 que llama a la función load_books_tags() (creada en la lógica).
```

```
    # Esta función carga los tags de los libros en el catalogo.
```

```
# Puede guiarse de las opciones 1 y 2.  
# Imprima el total de tags de los libros cargados.
```

```
elif int(inputs[0]) == 3:  
    print("Cargando información de tags de los libros....")  
    booksTags = load_books_tags(app)  
    print("Total de tags de los libros cargados: " + str(booksTags) + "\n")
```

```
elif int(inputs[0]) == 0:  
    working = False  
    print("\nGracias por utilizar el programa.")
```

```
else:  
    print("Opcion erronea, vuelva a elegir.\n")  
sys.exit(0)
```

Y finalmente aqui esta el error al poner opcion 1 en la consola corriendo el codigo:

```
(base) nikki@MacBook-Air-de-Nicole ISIS1225-Laboratorio-1 % clear  
  
(base) nikki@MacBook-Air-de-Nicole ISIS1225-Laboratorio-1 % /usr/local/bin/python3  
"/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-  
1/main.py"
```

Opciones:

1- Cargar Libros

2- Cargar Tags

3- Cargar Tags de libros

0- Salir

Seleccione una opción para continuar

1

Cargando información de libros....

Traceback (most recent call last):

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/main.py", line 11, in <module>

```
main()
```

```
~~~~^^
```

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/main.py", line 6, in main

```
view.main()
```

```
~~~~~^^
```

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/view.py", line 125, in main

```
books = load_books(app)
```

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/view.py", line 75, in load\_books

```
books = logic.load_books(app,
```

```
    "GoodReads/books-small.csv")
```

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/logic.py", line 100, in load\_books

```
    return book_size(catalog)
```

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/logic.py", line 167, in book\_size

```
    books_size = catalog["book"]
```

~~~~~^ ^ ^ ^ ^ ^ ^ ^

KeyError: 'book'

(base) nikki@MacBook-Air-de-Nicole ISIS1225-Laboratorio-1 %

y aqui esta el error de la opcion 3: (base) nikki@MacBook-Air-de-Nicole ISIS1225-Laboratorio-1 % /usr/local/bin/python3 "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/main.py"

Opciones:

1- Cargar Libros

2- Cargar Tags

3- Cargar Tags de libros

0- Salir

Seleccione una opción para continuar

3

Cargando información de tags de los libros....

Traceback (most recent call last):

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/main.py", line 11, in <module>

main()

~~~~^ ^

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/main.py", line 6, in main

view.main()

~~~~~^ ^

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/view.py", line 141, in main

booksTags = load\_books\_tags(app)

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/view.py", line 101, in load\_books\_tags

```
booksTags = logic.load_books_tags(app,  
                                   "GoodReads/book_tags-small.csv")
```

File "/Users/nikki/Documents/2026-2/EDA/Repositorio GitHub/ISIS1225-Laboratorio-1/App/logic.py", line 151, in load\_books\_tags

```
return book_tags(catalog)
```

TypeError: 'dict' object is not callable

(base) nikki@MacBook-Air-de-Nicole ISIS1225-Laboratorio-1 %

## Respuesta de la IAG

SÍ 🤔 y de hecho los dos errores tienen causas bastante claras. No parece que el problema sea el archivo CSV ni el `view`: son **nombres de claves y variables que no coinciden con lo que creaste en `new_logic()`**.

Vamos uno por uno.



## Error de la opción 1

El error importante es este:

```
KeyError: 'book'
```

Y Python te muestra exactamente dónde ocurre:

```
def book_size(catalog):  
    books_size = catalog["book"]  
    return set.size(catalog["books"])
```

El problema está en:

```
books_size = catalog["book"]
```

**¿Por qué?**

Cuando creaste tu catálogo en `new_logic()`, las claves que existen son:

```
catalog = {
    "books": None,
    "tags": None,
    "book_tags": None,
}
```

Mira la diferencia:

### Existe en tu catálogo Estás intentando buscar

|                                                                                               |                                                                                               |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| "books"      | "book"       |
| "tags"       | "tags"       |
| "book_tags"  | "book_tags"  |

Entonces Python intenta hacer:

```
catalog["book"]
```

pero **no existe una clave llamada "book"**.

Por eso:

```
KeyError: 'book'
```

### ¿Cómo arreglarlo?

Realmente esa línea ni siquiera la necesitas.

Tu función puede quedar:

```
def book_size(catalog):
    return set.size(catalog["books"])
```

O, si quieres mantener la variable:

```
def book_size(catalog):
    books_size = catalog["books"]
    return set.size(books_size)
```

Yo usaría la primera porque es más sencilla.

### Entonces:

```
def book_size(catalog):
    return set.size(catalog["books"])
```

Y listo, la opción 1 debería dejar de dar ese `KeyError`.

---



## Ahora el error de la opción 3

Este es un poquito más interesante:

```
TypeError: 'dict' object is not callable
```

Y me dices que en tu versión actual aparece:

```
return book_tags(catalog)
```

Aquí hay una pista MUY importante.

Tienes esto en `new_logic()`:

```
catalog = {
    "books": None,
    "tags": None,
    "book_tags": None,
}
```

Es decir, `book_tags` es **una clave del diccionario**, no una función.

Pero en algún punto estás haciendo:

```
book_tags(catalog)
```

Los paréntesis `()` significan:

"Python, ejecuta esto como si fuera una función".

Pero `book_tags` está siendo interpretado como un diccionario/estructura de datos, no como una función.

Por eso:

```
TypeError: 'dict' object is not callable
```

---



## PERO hay otro problema en tu `load_books_tags`

En el código que me pegaste originalmente tienes:

```
def load_books_tags(catalog, filename):

    book_tags = catalog.get("Book_tags")
    book_tags_file = os.path.join(data_dir, filename)
    catalog["Books_tags"] = set.load_set(book_tags, book_tags_file)

    if book_tags is None:
        return None
    else:
        return set.size(catalog.get("Book_tags"))
```

Aquí tienes **varios errores de nombres**.

Fíjate:

En `new_logic()`:

`"book_tags"`

Pero después escribiste:

`"Book_tags"`

y:

`"Books_tags"`

Python diferencia mayúsculas y minúsculas.

Por lo tanto:

`"book_tags"`

es diferente de:

`"Book_tags"`

y diferente de:

`"Books_tags"`



## Cómo debería quedar `load_books_tags`

Siguiendo exactamente el mismo patrón que utilizaste para `load_books()` y `load_tags()`, yo la dejaría así:

```
def load_books_tags(catalog, filename):
```

```

"""
Carga los tags de los libros del archivo y los agrega
al conjunto book_tags del catalogo.

:param catalog: Catalogo de la aplicación
:type catalog: dict
:param filename: Nombre del archivo csv con los tags de los libros
:type filename: str

:returns: Tamaño del conjunto de tags de los libros
:rtype: int
"""

book_tags = catalog["book_tags"]
book_tags_file = os.path.join(data_dir, filename)

catalog["book_tags"] = set.load_set(book_tags, book_tags_file)

if set.is_empty(catalog["book_tags"]):
    return 0
else:
    return set.size(catalog["book_tags"])

```

## Fíjate en la lógica:

Primero obtenemos el conjunto:

```
book_tags = catalog["book_tags"]
```

Luego construimos la ruta:

```
book_tags_file = os.path.join(data_dir, filename)
```

Después cargamos los datos:

```
catalog["book_tags"] = set.load_set(book_tags, book_tags_file)
```

Y finalmente preguntamos si quedó vacío:

```
if set.is_empty(catalog["book_tags"]):
    return 0
```

o devolvemos su tamaño:

```
return set.size(catalog["book_tags"])
```



## Hay una idea MUY importante aquí

Tu catálogo funciona como una caja con tres compartimentos:

```
catalog = {  
    "books": ...,  
    "tags": ...,  
    "book_tags": ...  
}
```

Así que **cada vez que quieras acceder a uno de ellos tienes que usar exactamente el mismo nombre.**

**Correcto** 

```
catalog["books"]  
catalog["tags"]  
catalog["book_tags"]
```

**Incorrecto** 

```
catalog["book"]  
catalog["Book_tags"]  
catalog["Books_tags"]
```

---

## También encontré otro error que te va a aparecer

En tu `load_tags()` tienes:

```
tags = catalog["tags"]  
tagsfile = os.path.join(data_dir, filename)  
catalog["tags"] = set.load_set(tags, tagsfile)  
  
if set.is_empty(tags):  
    return 0  
else:  
    return set.size(tags)
```

Aquí hay un detalle.

Después de:

```
catalog["tags"] = set.load_set(tags, tagsfile)
```

**tags sigue apuntando al conjunto anterior**, no necesariamente al conjunto que acabas de cargar.

Es más seguro hacer:

```
def load_tags(catalog, filename):
    tags = catalog["tags"]
    tagsfile = os.path.join(data_dir, filename)

    catalog["tags"] = set.load_set(tags, tagsfile)

    if set.is_empty(catalog["tags"]):
        return 0
    else:
        return set.size(catalog["tags"])
```

Esto es el mismo patrón que te recomiendo para `book_tags`.

---



## Y hay otra cosita en `data_dir`

Tienes:

```
data_dir = os.path.dirname(os.path.realpath('**file**')) + '/Data/'
```

Si eso está **literalmente así en tu archivo**, está mal.

Normalmente debería ser:

```
data_dir = os.path.dirname(os.path.realpath(__file__)) + '/Data/'
```

Es decir, `__file__` con **dos guiones bajos antes y después**.

Esto sirve para obtener la ubicación del archivo `logic.py` y luego encontrar la carpeta `Data`.

---



## Y ojo con la ruta del View

Tienes:

```
books = logic.load_books(app,
                          "GoodReads/books-small.csv")
```

y:

```
tags = logic.load_tags(app,
                       "GoodReads/tags.csv")
```

y:

```
booksTags = logic.load_books_tags(app,  
                                   "GoodReads/book_tags-small.csv")
```

Pero en `logic.py` haces:

```
booksfile = os.path.join(data_dir, filename)
```

Por lo tanto, si `data_dir` ya termina en:

```
/Data/
```

estarías intentando buscar algo como:

```
Data/GoodReads/books-small.csv
```

**Eso podría ser otro error después, dependiendo de cómo tengas organizada tu carpeta.**

Si tu estructura es:

```
ISIS1225-Laboratorio-1/  
├── App/  
│   ├── logic.py  
│   └── view.py  
├── Data/  
│   └── GoodReads/  
│       ├── books-small.csv  
│       ├── tags.csv  
│       └── book_tags-small.csv  
└── main.py
```

entonces tendría sentido que el `filename` fuera simplemente:

```
"GoodReads/books-small.csv"
```

**si `data_dir` apunta a `Data/`.**



## Lo que yo corregiría AHORA MISMO

En tu `logic.py`, mínimo cambia estas tres cosas:

### 1. `data_dir`

```
data_dir = os.path.dirname(os.path.realpath(__file__)) + '/Data/'
```

## 2. book\_size

```
def book_size(catalog):  
    return set.size(catalog["books"])
```

## 3. load\_books\_tags

```
def load_books_tags(catalog, filename):  
  
    book_tags = catalog["book_tags"]  
    book_tags_file = os.path.join(data_dir, filename)  
  
    catalog["book_tags"] = set.load_set(book_tags, book_tags_file)  
  
    if set.is_empty(catalog["book_tags"]):  
        return 0  
    else:  
        return set.size(catalog["book_tags"])
```

Y además revisaría load\_tags:

```
def load_tags(catalog, filename):  
  
    tags = catalog["tags"]  
    tagsfile = os.path.join(data_dir, filename)  
  
    catalog["tags"] = set.load_set(tags, tagsfile)  
  
    if set.is_empty(catalog["tags"]):  
        return 0  
    else:  
        return set.size(catalog["tags"])
```



## En resumen

Tus dos errores vienen de cosas diferentes:

### Opción 1:

```
KeyError: 'book'
```



Estás buscando "book" pero tu catálogo tiene "books".

### Opción 3:

```
TypeError: 'dict' object is not callable
```



Estás intentando hacer algo como:

```
book_tags(catalog)
```

como si `book_tags` fuera una función, cuando en realidad es una estructura/dato del catálogo. Además, en tu código hay inconsistencias entre `"book_tags"`, `"Book_tags"` y `"Books_tags"`.

Y una recomendación importante: **no cambies todo el proyecto todavía**. Primero corrige esas partes, corre nuevamente las opciones **1, 2 y 3**, y si aparece otro error, mándame el nuevo traceback. Así podemos ir siguiendo la cadena de errores uno por uno sin romper cosas que ya funcionan. 🤖🖥️